

面向对象程序设计实验报告

基于 pygame 的"飞机大战"小游戏

姓名： _ 许航 _ 学号： _ 2024082125 _ 班级： _ 软件252班 _ 日期： _ 2026/9/28 _

一、实验目的

1. 掌握 Python 中类的定义、构造方法 `__init__`、实例属性的使用，能自己把游戏里的角色抽象成一个个"类"。
2. 理解**继承**：把玩家、敌机、子弹重复的坐标 / 图片 / 速度 / 绘制代码抽到基类 `SpriteBase`，子类只写自己的特殊部分，避免同样的代码写三遍。
3. 理解**方法重写**：基类的 `move()` 只留一个空方法占位，三个子类各自重写成完全不同的运动方式，主循环里却可以统一用 `obj.move()` 调用。
4. 理解**封装**：把数据（坐标、速度）和操作它们的方法放在同一个类里，外部不去直接改对象的内部状态。
5. 掌握**静态方法**和 **@property** 的适用场景，知道什么时候该用 `@staticmethod`、什么时候用属性而不是普通方法。
6. 能够用 pygame 完成一个可交互的小游戏：主循环、事件处理、矩形碰撞检测、图片资源加载、帧率控制。

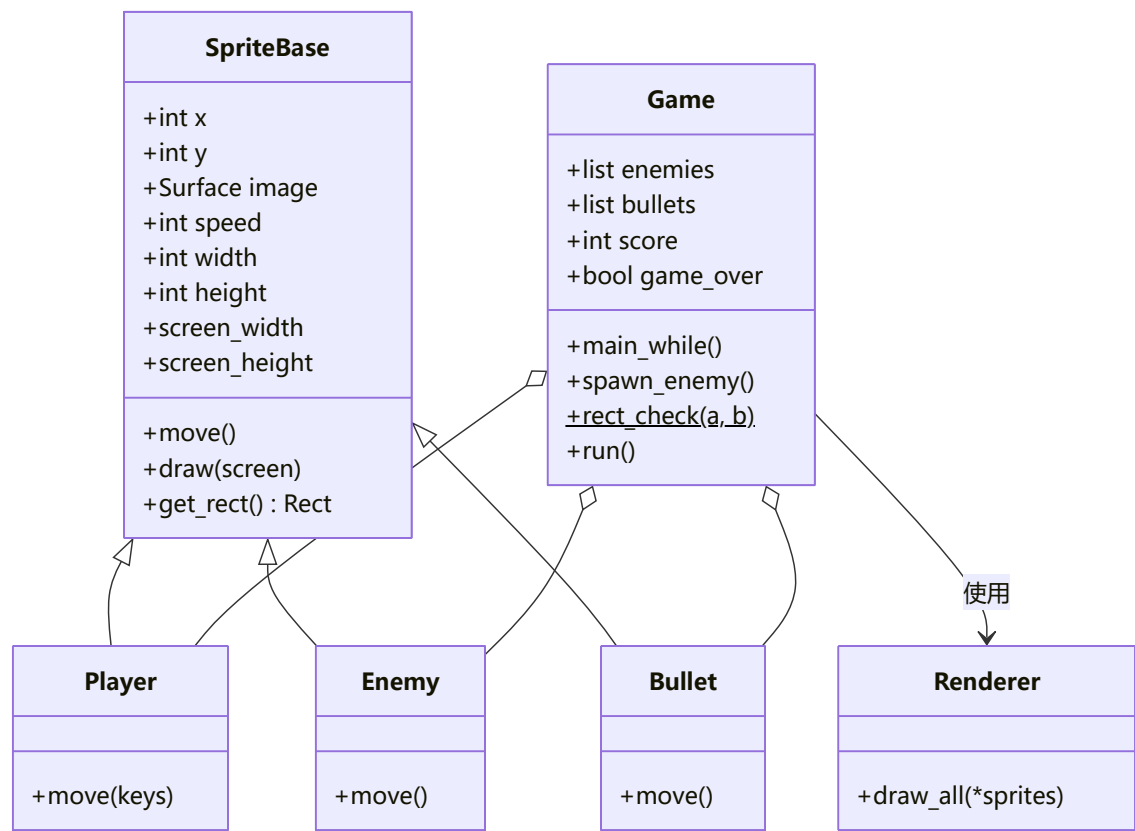
二、实验环境

项目	内容
操作系统	Windows
编程语言	Python 3.14.7
虚拟环境	项目自带 .venv（用 uv 创建， <code>include-system-site-packages = false</code> ）
图形库	pygame-ce 2.5.8（pygame 的社区版，用法与 pygame 完全一致，代码里照样 <code>import pygame</code> ）
开发工具	VSCode + Jupyter 扩展
图像素材	Kenney 的 Pixel Shmup 素材包（Player.png / Enemy.png / Bullet.png）

另外 `Game` 里还有一个静态方法 `rect_check(a, b)`，用来判断两个实体是否碰撞。

一句话概括继承关系：`Player`、`Enemy`、`Bullet` 都继承自 `SpriteBase` (`class Player(SpriteBase)`)，三个子类只需要写自己的 `__init__` 和 `move()`，其余全部从基类继承。

3.3 类图 (Mermaid 版)



3.4 各类职责一览

类名	职责	主要属性	主要方法
SpriteBase	基类，抽出所有实体的公共部分	x, y, image, speed, screen, width, height	move() (空实现)、draw()、get_rect()、screen_width / screen_height
Player	玩家飞机：键盘左右移动，越界穿越	从基类继承	move(keys) (重写)

类名	职责	主要属性	主要方法
Enemy	敌机：屏幕顶部外随机出现，向下匀速移动	从基类继承	<code>move()</code> （重写）
Bullet	子弹：从玩家机头发出，向上移动	从基类继承	<code>move()</code> （重写）
Renderer	渲染器，只负责把精灵画到屏幕上	screen	<code>draw_all(*sprites)</code>
Game	主控类：窗口、主循环、生成敌机、碰撞判定、计分、结束	screen, clock, player, enemies, bullets, score, game_over, spawn_timer, spawn_interval, font, renderer	<code>__init__</code> 、 <code>main_while()</code> 、 <code>spawn_enemy()</code> 、 <code>rect_check()</code> （静态方法）、 <code>run()</code>

四、各类详细设计

4.1 SpriteBase —— 基类，抽公共部分

这是整个程序里最关键的一个类。我先把三个角色"必须有"的东西列出来：位置 (x, y)、图片 (image)、速度 (speed)、要画到哪个屏幕 (screen)、图片的宽高 (width, height)。这些是共有的，所以全部放进基类的 `__init__` 里一次性收拢。

方法设计上的取舍：

- `draw(screen)`：三个子类的画法完全一样（都是 `screen.blit()`），所以实现写在基类，子类一个字都不用写。这就是继承最直接的省事之处。
- `move()`：**故意写成空方法**（`pass`）。基类并不知道玩家怎么动、敌机怎么动，它只规定"每个实体都必须有一个 `move` 方法"，具体实现交给子类去重写。主循环里因此可以统一写 `self.player.move(keys)`、`enemy.move()`。
- `width / height` 在 `__init__` 里就从 `image.get_width()` 取出来存好，之后判断越界、算子弹出生位置时直接读属性，不用反复去问图片对象。
- `screen_width / screen_height` 用了 `@property`：窗口大小是从 `screen` 对象上取来的，不会变，写成属性之后外面用 `self.screen_width` 就像读一个变量一样，但它背后其实是一次方法调用。相比在 `__init__` 里存一个固定数字，这样写不会出现"窗口改了大小但属性还是旧值"的问题。

下面是基类的完整代码：

```
In [27]: class SpriteBase:                                # 实体基类
    def __init__(self, x, y, image, speed, screen):        # 实体本身、坐标、图像、速度
        self.x = x
        self.y = y
        self.image = image
        self.speed = speed
        self.screen = screen
        self.width = image.get_width()
        self.height = image.get_height()

    def move(self):
        pass                                              # 空实现，等子类重写

    def draw(self, screen):
        screen.blit(self.image, (self.x, self.y))

    def get_rect(self):                                  # 碰撞矩形
        rect = pygame.Rect(self.x, self.y, self.width, self.height)
        return rect.inflate(-self.width * 0.2, -self.height * 0.4)

    @property
    def screen_width(self):
        return self.screen.get_width()

    @property
    def screen_height(self):
        return self.screen.get_height()
```

4.2 Player —— 玩家飞机

Player 是 SpriteBase 的子类，它自己只需要负责三件事：算出出生的位置和速度、把图片缩放到合适大小、实现键盘控制。

构造里有个顺序上的坑：基类的 `__init__` 要用 `image` 去算 `width / height`，所以必须**先把图片加载、缩放好**，再调用 `super().__init__(...)`。我最初把 `super().__init__()` 写在最前面，结果 `self.image` 还没有，基类直接报错。调整后的写法是：先算 `x / y / speed`，再准备 `image`，最后才交给基类存起来。

几个细节：

- 出生位置 `x = self.screen_width // 2`、`y = self.screen_height // 3 * 2`，也就是底部偏上的正中间，用除法算而不是写死数字，窗口大小改了也不用跟着改。
- `speed = self.screen_width // 100`，速度跟窗口宽度成比例（1600 宽的窗口就是每帧 16 像素），这样画面变大时手感不会变得很慢。
- `move(keys)` 是**重写**基类的方法，参数从空变成了 `keys`：主循环里用 `pygame.key.get_pressed()` 拿到一张“当前哪些键被按住”的表，再交给玩家自己判断。同时按住方向键可以持续移动，不用监听按键事件。
- 越界穿越：`self.x < 0` 说明整个飞机从左边界出去了，就让它的 `x` 变成 `屏幕宽 - 飞机宽`（出现在右边界）；反过来右边出去了就回到 0。这里减去 `self.width` 是因为 `x` 存的是**左上角**坐标，不减去的话飞机会有一半留在屏幕外。

4.3 Enemy —— 敌机

`Enemy` 的构造函数参数是 `x, y, speed, screen`，也就是说**敌机的位置和速度由外面传进来**（实际是 `Game.spawn_enemy()` 决定的），它自己只负责把图片处理成“朝下飞”的样子：`transform.scale` 缩到 `50×50`，再用 `transform.rotate(image, 180)` 旋转 `180°`，因为素材里的飞机机头是朝上的。

`move()` 只改 `y`：`self.y += self.speed`，向下匀速。敌机不需要读键盘，所以它的 `move` 不带参数——这正好体现了重写的好处：**父类只规定方法名，每个子类的参数和实现都可以不一样，主循环按自己的需要调用即可。**

4.4 Bullet —— 子弹

子弹和敌机刚好相反，`move()` 里是 `self.y -= self.speed`，向上飞。它的速度是 8（敌机是 3），比敌机快才能在敌机落下之前追上它。

子弹的出生位置由 `Game` 里算：`position = self.player.x + (self.player.width // 2) - 3`，也就是玩家飞机的水平中点上。减 3 是目测微调——子弹图虽然缩到了 `20×20`，但有效像素比较窄，不减一点看起来会偏出机头。

发射的时机用键盘**按下事件**（`KEYDOWN + K_SPACE`）控制，按一次发一颗，不用自己写冷却时间。

4.5 Renderer —— 渲染器

我单独拆了一个 `Renderer` 类出来，就是为了让 `Game` 里只剩下游戏逻辑，画图的事情全部由它负责。

它的 `draw_all(self, *sprites)` 用了**可变参数**，配合 `isinstance` 判断：如果传进来的是列表或元组就遍历着画，如果是一个单独的精灵就直接画。这样在 `Game` 里可以一行搞定：

```
self.renderer.draw_all(self.enemies, self.player, self.bullets)
```

一个列表加两个单个对象混着传也不用在外面写循环，参数个数还可以随便加。渲染器自身的代码如下：

```
In [28]: class Renderer:
          def __init__(self, screen):
              self.screen = screen

          # 更新每个实体在屏幕上的位置
          def draw_all(self, *sprites):
              for item in sprites:
                  if isinstance(item, (list, tuple)):
                      for sprite in item:
                          sprite.draw(self.screen)
```

```
else:
    item.draw(self.screen)
```

4.6 Game —— 主控类

`Game` 是唯一一个"不继承任何东西"的类，它更像一个总管，管的是**游戏规则**而不是某个实体。它的属性可以分成几组：

分组	属性	说明
窗口与时间	<code>screen</code> , <code>clock</code> , <code>window_width=1600</code> , <code>window_height=900</code>	创建窗口， <code>clock</code> 用来控制帧率
实体	<code>player</code> , <code>enemies</code> (列表), <code>bullets</code> (列表)	玩家只有一个；敌机和子弹都不止一个，所以用列表存
状态	<code>game_over</code> , <code>score</code>	对局是否结束、当前分数
生成控制	<code>spawn_timer</code> , <code>spawn_interval=60</code>	帧计数器与生成间隔
显示	<code>font</code> , <code>renderer</code>	分数用的字体、负责绘制的渲染器

`main_while()` 是整个游戏真正跑起来的地方，一个 `while` 循环每一帧做这些事，顺序很重要：

1. 处理事件：`pygame.event.get()` 里判断是不是点了关闭窗口（QUIT）、是不是按了空格（发射子弹）；
2. 用黑色填充屏幕（清掉上一帧的画面）；
3. 读键盘状态，让玩家移动；
4. 调用 `spawn_enemy()` 决定这一帧要不要生成敌机；
5. 遍历列表，让每架敌机、每颗子弹各自 `move()`；
6. 做两次碰撞检测：子弹打敌机（加分并删除双方）、玩家撞敌机（游戏结束）；
7. 清理飞出屏幕的子弹和敌机；
8. 交给 `renderer` 把所有精灵画出来，再画左上角的分数，最后 `flip()` 把这一帧显示出来；
9. `self.clock.tick(60)` 让循环稳定在 60 帧/秒。

顺序上的两点考虑：一是必须"先清屏、再移动、最后画"，否则会出现残影；二是碰撞检测放在移动之后，否则会出现"子弹明明碰到了却没反应"。另外，碰撞检测我特意没有写在子弹的移动循环里——因为撞上之后要同时从两个列表里删元素，写在移动的循环里容易把正在遍历的列表改掉。

`rect_check(a, b)` 是一个**静态方法**：它做的事情纯粹是"比较两个实体的矩形有没有重叠"，既不需要 `self`（不访问 `Game` 的任何属性），也不修改两个对象的状态。所以用 `@staticmethod` 最合适，调用时写 `self.rect_check(bullet, enemy)` 就行。

`run()` 是游戏的对外入口：先进主循环，循环因为 `game_over` 结束之后，画一屏"游戏结束 + 最终得分"，用 `pygame.time.wait(2000)` 停留 2 秒让玩家看清楚分数，然后释放资

源退出。

主程序入口是标准写法：`Game().run()`——只在这一行创建对象并启动，其余逻辑全部封装在类里面。

五、关键功能实现

5.1 碰撞检测

两个实体是否碰撞，本质上就是**两个矩形有没有重叠**，pygame 已经把这件事做好了（`colliderect`），我要做的是给每个实体算出自己的矩形。

这里有个必须处理的问题：**图片是正方形的，但飞机图案只占中间一块，四周都是透明像素**。如果直接用整张图的矩形去判定，会出现“还差一点才碰到就判定撞上了”的误判。所以在 `get_rect()` 里用 `inflate` 把矩形缩小：宽度缩 20%、高度缩 40%（上下方向的透明区域更大，所以缩得更多）。

```
In [29]: # 摘自 main.py: SpriteBase.get_rect()
class SpriteBase:
    # ..... 其余属性和方法此处省略

    def get_rect(self):                # 碰撞矩形
        rect = pygame.Rect(self.x, self.y, self.width, self.height)
        return rect.inflate(-self.width * 0.2, -self.height * 0.4)
```

有了每个实体的矩形，判断碰撞就只需要把两个矩形交给 `colliderect`。我把这个判断写成了 `Game` 的静态方法，这样无论要比较哪两个实体，都有同一个地方可以直接调用：

```
In [30]: # 摘自 main.py: Game.rect_check (静态方法)
class Game:
    # ..... 其余属性和方法此处省略

    @staticmethod
    def rect_check(a: SpriteBase, b: SpriteBase):
        if a.get_rect().colliderect(b.get_rect()):
            return True
        else:
            return False
```

主循环里一共用了两次：

```
for bullet in self.bullets[:]:          # 子弹打敌机
    for enemy in self.enemies:
        if self.rect_check(bullet, enemy):
            self.bullets.remove(bullet)
            self.enemies.remove(enemy)
            self.score += 10              # 每击落一架加 10 分
            break                          # 一颗子弹只打掉一架

for enemy in self.enemies:              # 玩家撞敌机
```

```

    if self.rect_check(self.player, enemy):
        self.game_over = True          # 对局结束
        break

```

写成静态方法的好处是：子弹、敌机、玩家可以随便组合着传进去，不用为"玩家撞敌机"再单独写一个方法——因为这段逻辑本来就只跟几何关系有关，跟 Game 的状态无关。

5.2 敌机生成计时

敌机不能每帧都生成（60 帧一秒的话屏幕会瞬间被塞满），所以用一个 `spawn_timer` 计数器配合 `spawn_interval = 60`：每过一帧计数器加 1，加到 60 就归零并生成一架敌机。

这里我用的是**帧计数**而不是记录真实时间，原因是主循环最后有 `clock.tick(60)` 把帧率固定在 60 帧/秒，那么 60 帧就正好等于 1 秒，不用每次去算时间差，简单又不容易出错。

敌机的 x 在窗口宽度范围内随机，y 固定为 -20——让它从**屏幕外面**开始往下掉，看起来就像从画面外飞进来的，而不会"凭空出现在屏幕边缘"。

```

In [31]: # 摘自 main.py: Game.spawn_enemy()
class Game:
    # ..... 其余属性和方法此处省略

    def spawn_enemy(self):
        self.spawn_timer += 1
        if self.spawn_timer >= self.spawn_interval:
            self.spawn_timer = 0
            x = random.randint(0, self.window_width - 40)          # 随机位置
            self.enemies.append(Enemy(x, -20, 3, self.screen))      # 窗口外出现

```

5.3 越界清理

子弹一直往上飞、漏掉的敌机一直往下掉，如果不处理，`bullets` 和 `enemies` 这两个列表会越变越长，最后内存占用变高、帧率掉下来。所以每帧检查一次，飞出屏幕的子弹和敌机直接从列表里删掉。

删除的时候**不能直接遍历原列表**，因为边遍历边 `remove` 会造成索引错位、漏掉元素甚至报错。这里的写法是遍历列表的**副本**（`self.bullets[:]`），但删除的是原列表里的元素：

```

In [32]: # 摘自 main.py: main_while() 中的越界清理
class Game:
    def main_while(self):
        # ..... 其余逻辑此处省略

        for bullet in self.bullets[:]:
            if bullet.y + bullet.height < 0:          # 子弹飞出顶部
                self.bullets.remove(bullet)
        for enemy in self.enemies[:]:
            if enemy.y > self.window_height:          # 敌人飞出底部
                self.enemies.remove(enemy)

```

注意两个判断的边界不一样：子弹要整个身体都出了顶部才删（`y + bullet.height < 0`），敌机则是**顶部一过屏幕下边缘**就删（`y > self.window_height`）——敌机掉到屏幕外面之后就没有任何作用了，早点清掉更省性能。

5.4 中文字体显示

pygame 的默认字体**不包含中文**，如果直接用 `pygame.font.SysFont(None, 30)` 渲染中文，屏幕上会出现一串方框（俗称豆腐块）。而这个程序的分数显示里带中文（“分数：10”），所以字体问题必须解决。

我的做法是**优先用项目自带的字体文件，再兜底去系统里找**：

- 先用 `os.path.exists()` 判断本地 ttf 文件在不在，在就用 `pygame.font.Font()` 直接加载——这样换台电脑只要带着字体文件，显示效果完全一样；
- 找不到本地字体时走 `match_font()` 兜底，它支持用逗号分隔的候选名字列表，会按顺序去找系统里装没装：微软雅黑（`microsoftyahei/msyh`）→ 黑体（`simhei`）→ 等线（`dengxian`）→ 宋体（`simsun`）；
- 整段用 `try / except` 包起来，万一两种情况都没找到，程序也只是字体不好看，而不会直接崩掉。

```
In [33]: # 摘自 main.py: Game.__init__() 中的字体加载
class Game:
    def __init__(self):
        # ..... 其余属性此处省略

        local_ttf = "字小魂方块黑(商用需授权).ttf"
        try:
            if os.path.exists(local_ttf):
                self.font = pygame.font.Font(local_ttf, 30)
            else:
                raise FileNotFoundError("未找到指定的本地字体")
        except Exception:
            # 兜底：按顺序在系统字体里找中文字体
            self.font = pygame.font.match_font("microsoftyahei,msyh,simhei,dengxian")
```

六、测试结果

测试方法：在 VSCode 里用 `.venv` 解释器直接运行 `main.py`，打开窗口后按功能点逐个操作，观察画面表现和控制台输出。每个功能点测 3 次以上，确认表现稳定。

序号	测试项	操作	预期结果	实际结果
1	玩家左右移动	按住 ← 或 →	飞机随之移动，按住不放能持续移动	与预期一致
2	越界穿越	一直往左（右）开	从一边出去后从另一边出现，不会卡在边缘	与预期一致

序号	测试项	操作	预期结果	实际结果
3	发射子弹	按空格	机头位置生成一颗向上的子弹，按一次发一颗	与预期一致
4	敌机生成	不动，观察 10 秒左右	约每秒出现一架，x 随机、机头朝下、匀速下落	与预期一致
5	击中加分	瞄准敌机开火	子弹与敌机同时消失，左上角分数 +10，控制台打印得分	与预期一致
6	碰撞结束	不躲，让敌机撞上玩家	立即结束对局，显示"游戏结束 得分：XX"，停留约 2 秒后窗口关闭	与预期一致
7	中文字体	观察左上角分数	显示"分数：0"等中文，没有方框乱码	与预期一致
8	关闭窗口	点右上角 ×	程序正常退出，PyCharm / VSCode 控制台无报错	与预期一致

运行截图

图 1 游戏运行中：玩家飞机、下落中的敌机、左上角实时分数

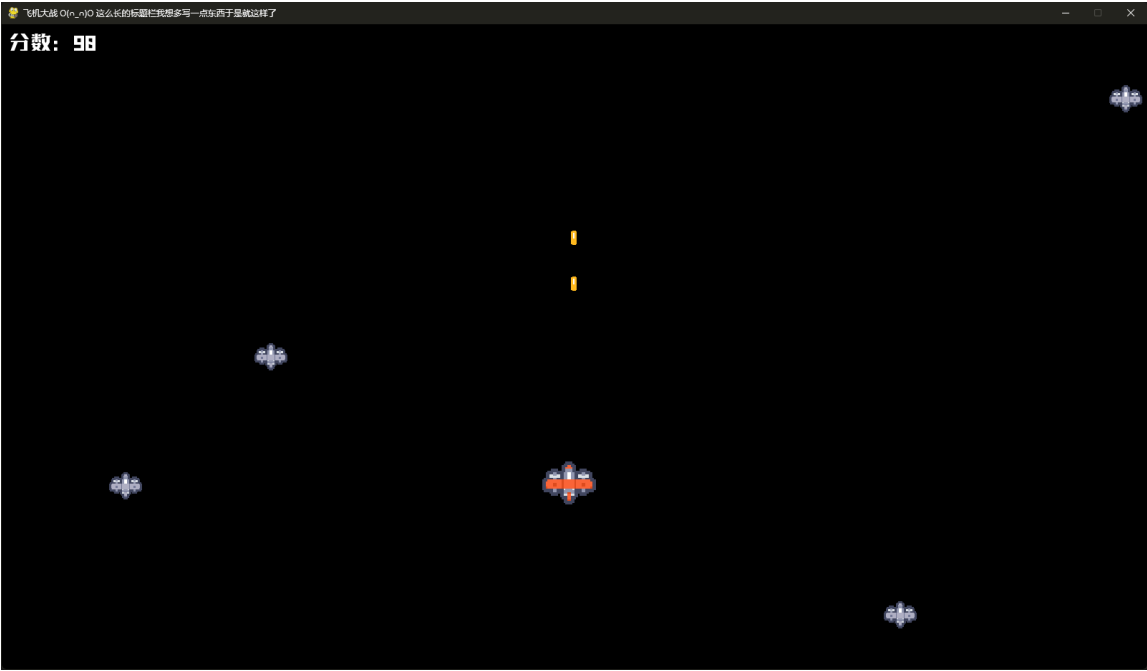


图 2 游戏结束画面："游戏结束 得分：210"



图 3 控制台输出：每次得分时打印的"得分：10 / 20 / 30 ..."

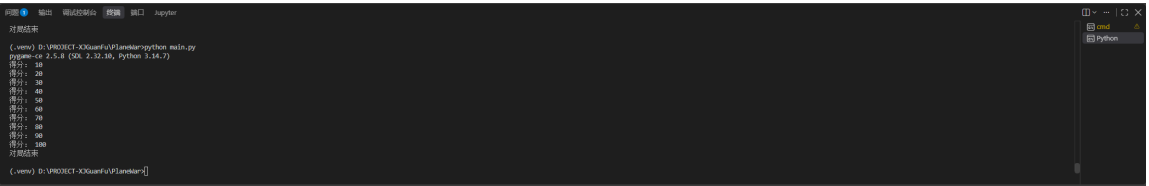
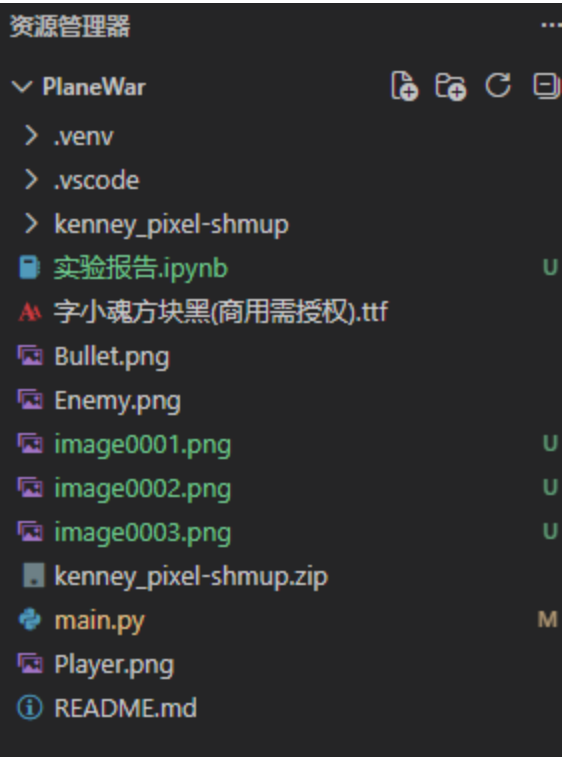


图 4 (可选) 工程结构：main.py、.venv、图片素材、字体文件



七、遇到的问题与解决

问题 1：分数里的中文显示成方框

- 现象：一开始用 `pygame.font.SysFont(None, 30)` 创建字体，画面上"分数：0"几个字全变成了小方块。
- 原因：pygame 自带的默认字体里没有中文字形，遇到没有的字就画成方框。
- 解决：改成优先加载项目目录下的中文字体文件 `pygame.font.Font("字小魂方块黑.ttf", 30)`；再用 `match_font()` 按"微软雅黑 → 黑体 → 等线 → 宋体"的顺序去系统里兜底找。用相对路径加载的要求是：**必须用 VSCode 打开 PlaneWar 这个文件夹本身**，否则工作目录不对会报找不到字体文件。

问题 2：明明还没碰到就判定碰撞了

- 现象：子弹从敌机旁边飞过去，分数却莫名其妙增加了；玩家飞机和敌机还隔着一段距离就结束了。
- 原因：碰撞用的是图片的矩形，而飞机素材是正方形图片，真正的图案只占中间一部分，四周全是透明像素，矩形把透明区域也算进去了。
- 解决：在 `get_rect()` 里用 `inflate()` 把矩形缩小后再判定——宽度缩 20%、高度缩 40%（上下方向的透明部分更多）。缩小之后碰撞判定跟肉眼看到的就基本一致了。

问题 3：游戏结束的瞬间报错（列表在遍历中被修改）

- 现象：子弹打中敌机的瞬间偶尔报错，或者一颗子弹明明打中了，敌机却没消失。
- 原因：最初把"碰撞检测"写在了 `for bullet in self.bullets:` 里面，删除元素的同时还在遍历这个列表，索引就错位了。
- 解决：两处修改——一是把碰撞检测从移动循环里单独拿出来；二是遍历的时候用列表的副本 `self.bullets[:]`，删除的是原列表，源码里注释也写明了这一点。另外打中之后加了一句 `break`，避免一颗子弹删掉多架敌机。

问题 4：`super().__init__()` 写太早，基类取不到 width / height

- 现象：玩家类一创建就报错，提示图片对象不存在。
- 原因：基类的 `__init__` 里要用 `image.get_width()` 来算 `width / height`，而我把 `super().__init__()` 写在了加载图片之前。
- 解决：把顺序调过来——先算好 `x / y / speed`、加载并缩放好图片，最后才交给 `super().__init__(...)` 统一保存。这也让我明白：子类调用父类构造方法之前，父类需要的参数必须已经准备好。

问题 5：窗口 1600×900 下，速度写死了手感很怪

- 现象：把玩家速度写成固定值后，换个窗口尺寸就变得特别快或者特别慢。
- 原因：速度是写死的数字，没有跟画面大小关联。
- 解决：改成按比例算——玩家速度 `speed = self.screen_width // 100`，出生位置也用 `screen_width / screen_height` 来算，这样改窗口大小时不用去挨个调数字。

问题 6: pygame 和 pygame-ce 混装

- 现象：环境里同时装了 pygame-ce 和 pygame 时，`import pygame` 到底导入哪个会不确定，报错信息也让人困惑。
- 原因：两者用的是同一个包名，装在一个环境里会互相覆盖。
- 解决：虚拟环境里只装 pygame-ce (`pip list` 已确认没有 pygame)，因为 pygame-ce 是 pygame 的社区维护版本，API 完全一样，我们代码里照样 `import pygame`，不用改任何代码。

八、总结

这次实验做完，收获最大的地方是“先抽象再写代码”这件事本身。

最开始我是按照“玩家、敌机、子弹”三个东西分别往下写的，三份代码里有一大堆重复：坐标、图片、速度、绘制、碰撞矩形……改一个地方要改三遍。后来强迫自己停下来，先问了一句“这三个东西共同的是什么、不一样的又是什么”，才把 `SpriteBase` 抽出来：

- **继承**带来的好处是实打实的：`draw()`、`get_rect()`、坐标属性的初始化都只写了一遍，三个子类加起来省掉了三四十行重复代码；`width / height` 由基类从图片里取，子类完全不用操心。
- **方法重写**让“同一个 `move()` 有不同行为”变成可能：主循环里不需要判断“这是谁”，统一调用 `move()` 就行，玩家读键盘、敌机向下、子弹向上各干各的。以后想加一种“斜着飞的敌机”，只要派生一个新子类重写 `move()`，主循环一行都不用改。
- **静态方法**的使用场景也搞清楚了：`rect_check(a, b)` 只做两个矩形之间的比较，不依赖任何对象的状态，所以用 `@staticmethod`；而 `draw_all()` 需要知道自己往哪个屏幕上画，就必须是实例方法。
- **封装**的体会是：外部（主循环）只通过 `move()`、`draw()`、`get_rect()` 这些方法去操作对象，不去直接改对象内部的细节；图片怎么缩放、透明区域怎么避开，都是各类自己的事。
- 细节上还踩了几个坑，比如“遍历列表的同时删元素”和“`super().__init__()` 的调用时机”，都是写代码之前没想清楚导致的，现在印象很深。

不足和可以继续改进的地方：

1. 难度不会变化——敌机速度固定是 3，而且只有一种敌机。可以按分数提高生成频率、给敌机加速度，玩家玩起来才有挑战性；
2. 没有生命值，一碰就结束，也没有音效和开始 / 重新开始的界面；
3. 现在按一次空格发一颗子弹，可以加一个发射冷却时间做成连发，手感会更好；
4. 很多数字（速度 8、生成间隔 60、加分数 10）是直接写在代码里的，如果要长期维护，最好抽成类属性或者单独的配置常量，改起来更集中。

不过就这次实验来说，能把这六个类的关系理清楚、把面向对象的继承 / 重写 / 封装真正用上，而不是写成一堆面向过程的 if-else，是最大的进步。

附录：运行方式与代码统计

运行方式

- 1. 用 VSCode 打开 `PlaneWar` 文件夹（工作目录必须是这个文件夹，因为图片和字体用的是相对路径）；
- 2. 右下角把解释器切换成项目里的 `.venv`；
- 3. 打开 `main.py`，点右上角运行按钮，或者终端里执行：

`python main.py`

代码统计（`main.py` 单文件）

项目	数量
总行数	232 行
类	6 个（SpriteBase、Player、Enemy、Bullet、Renderer、Game）
基类 / 子类	1 个基类 + 3 个派生类
静态方法	1 个（Game.rect_check）
被重写的方法	3 个（三个子类各自的 move）
图片资源	Player.png、Enemy.png、Bullet.png